



CS 40700

Design Document

Team 16

Xiuwen “Charlie” Fang, Xinyi Guan, Yuchuan Huang, Xinyi Wu

Purpose

Our project aims to create a beginner-friendly and engaging platform that combines foundational programming education with the immersive and interactive elements of a role-playing game (RPG). This project addresses the challenges faced by students and programming novices when learning abstract concepts such as loops, conditionals, and pointers. By incorporating storytelling, gamification, and hands-on practice, Bareturn0; seeks to make programming accessible, enjoyable, and memorable. The game introduces core programming concepts through gamified, puzzle-based challenges that align with progressive educational goals, providing immediate feedback and rewards to reinforce learning. By integrating interactive elements like mini-games, quizzes, and problem-solving tasks into a narrative-driven experience, it ensures users learn by doing, bridging the gap between theory and application.

Bareturn0; features an engaging RPG framework where players navigate a visually appealing map, unlocking new levels and story chapters as they progress. Each chapter presents unique programming tasks tied seamlessly into the storyline, allowing players to guide a character through challenges using code. With a customizable code editor and interactive mechanics, the game adapts to users' skill levels, providing a smooth and scalable learning experience. Players are rewarded with achievements, in-game currency, and character upgrades for completing challenges and mastering concepts. A card-based combat system, tied to coding tasks, adds a strategic layer, fostering critical thinking and problem-solving skills. The reward system motivates players to explore optional levels, strengthen specific skills, and achieve ambitious goals.

The project caters to students, beginners in programming, and educators seeking innovative teaching tools. By gamifying the learning process, it appeals to users who prefer hands-on and interactive learning environments. The platform ensures scalability and usability, with plans for cross-platform compatibility and modular design for future expansions. Built with Unity, Bareturn0; provides a seamless, cross-platform gaming experience, while a secure backend using Node.js and MongoDB ensures robust user management and data storage. Supported by AWS for scalability and performance, Bareturn0; is designed to deliver a reliable and immersive platform capable of serving a diverse user base. By blending education with entertainment, Bareturn0; aims to redefine how programming is taught and learned, inspiring confidence and curiosity in learners of all ages and backgrounds.

Functional Requirements

- Welcome and Main Menu:
 1. As a user, I would like to see a loading animation when the game starts, so that the game feels engaging and polished.
 2. As a user, I want the option to skip the loading animation and directly access the main menu, so I can save time when re-entering the game.
 3. As a user, I would like to see a main page after loading the welcome animation, including the function buttons I need(log-in, settings, etc.).
 4. As a user, I would like to create my own account.
 5. As a user, I would like to log in and log out of my own account.
 6. As a user, I want to access settings to adjust volume and sound effects.
 7. As a user, I would like to view statistical data(percentage) on game progress, such as card collections and features collection progress.
- Save Management:
 8. As a user, I would like to create multiple save files under one account, so I can have different game progressions.
 9. As a user, I would like to load a specific save file from the list of my saves.
 10. As a user, I would like to delete save files that I no longer need.
- Achievements
 11. As a user, I would like to unlock achievements with dates of receiving for significant milestones in the game, so I feel a sense of progression and accomplishment.
 12. As a user, I want an on-screen notification when I unlock an achievement, so I feel immediate satisfaction and recognition.
 13. As a user, I would like to earn rewards (e.g., in-game currency, items, or cosmetics) for completing achievements, so I feel incentivized to pursue them.
 14. As a user, I want a dedicated page in the game to display all the achievements I have unlocked, so I can revisit and showcase my accomplishments.
- Game Background and Core Flow
 15. As a user, I would like the option to pause the game and return to the main menu, without saving the progress.
 16. As a user, I want to see an option to restart the current level from the pause menu, so I can quickly try again if I make a mistake.
 17. As a user, I want to see a detailed breakdown of rewards (e.g., experience points, loot) on the victory settlement screen, so I know what I've earned.
 18. As a user, I would like the failure screen to provide suggestions or tips for improving in the next attempt, so I feel motivated to try again.

19. As a user, I want the victory and failure screens to offer quick options to replay the level or return to the map screen, so I can easily decide my next step.
20. As a user, I would like a narrative story to provide background for the game world and context for my actions.
- User Page
 21. As a user, I would like to view my profile information in the user page.
 22. As a user, I want the ability to modify my personal details (e.g., username, email, and password) from the user page, so I can keep my account up to date.
 23. As a user, I want to upload and update a custom profile picture, so I can personalize my account.
 24. As a user, I want a quick link to contact support or provide feedback directly from my user page, so I can resolve issues or share suggestions conveniently.
- Card Game Mechanics
 25. As a user, I would like to preview card attributes to understand their effects before using them.
 26. As a user, I would like to select the functional cards I need before entering battle rounds to customize my strategy.
 27. As a user, I would like to filter and sort cards by attributes (e.g., attack, defense, or special effects) when selecting my functional cards before a battle, so I can quickly find the cards I need.
 28. As a user, I would like to save multiple custom decks for different battle scenarios, so I can switch strategies easily.
 29. As a user, I want both my character and the boss to have health points, so the game has a clear win or lose condition based on health depletion.
 30. As a user, I want each card I choose to reduce my stamina by a specified amount, so I need to carefully manage my stamina during each turn.
 31. As a user, I want my stamina to regenerate at the start of each round, so I can have a fresh pool of resources to play new cards.
 32. As a user, I want the game to operate in a turn-based format, so I have time to plan my strategy and respond to the boss's moves.
 33. As a user, I want each turn of the battle to be under a time count-down mode.
 34. As a user, I want to get new cards at the beginning of each round, so I have a fresh set of options for every turn.
 35. As a user, I want discarded cards to go into a discard pile, so my deck remains organized and cards are not reused immediately.
 36. As a user, I want the discard pile to be reshuffled into the draw pile when the draw pile is empty, so I can continue to play without interruption.

- 37. As a user, I want the boss to try to defeat me by playing cards strategically, so that the combat feels fair, challenging, and engaging.
- 38. As a user, I want the boss to have unique abilities or cards that challenge my strategy, so the battles feel varied and exciting.
- 39. As a user, I want to see a visual indication of changes in health, stamina, and card piles during the game, so I can easily track the current state of the battle.
- Map and Exploration
 - 40. As a user, I want to unlock new map areas by either completing specific levels or learning certain skills, so I can explore the game world in a way that aligns with my progress and choices.
 - 41. As a user, I want to see a figure of mine on the map to see which level I'm currently on.
 - 42. As a user, I want the map to restrict access to incomplete levels while allowing me to freely revisit previously completed levels, so I can replay them for practice or better rewards.
 - 43. As a user, I want to include optional levels in the map which focus on specific skills or learning topics, so I can choose to strengthen areas that interest me.
 - 44. As a user, I want to zoom in and out of the map with ease, so I can navigate between detailed sections and the overall layout effortlessly.
 - 45. As a user, I want to unlock map-themed achievements , so I feel motivated to explore thoroughly.
- Reward System
 - 46. As a user, I would like to earn rewards for completing levels and challenges, which can be used to exchange for blank functional cards or other upgrades.
 - 47. As a user, I would like to receive different types of rewards (e.g., blank cards, rare items, in-game currency, or exclusive abilities), so my gameplay feels more varied and engaging.
 - 48. As a user, I would like bonus rewards for completing levels with certain conditions (e.g., within a time limit or without losing health), so I can aim for more ambitious goals.
 - 49. As a user, I would like the rewards for challenges to scale with difficulty, so I feel incentivized to take on harder challenges.
- Learning Features
 - Learning printf:
 - 50. As a user, I would like to name my character while learning the printf function, so I can understand how to format strings and personalize my game experience.
 - 51. As a user, I want the game to have NPC interactions to teach me the basics of the printf function, so I can use them effectively.
 - 52. As a user, I would like mini-games to practice printf with customized exercises.

- 53. As a user, I want to earn my first achievement, "Person You Know Who," by demonstrating mastery of the printf function, so I feel motivated to continue learning.
- 54. As a user, I want NPCs to offer increasingly complex printf challenges (e.g., combining text with numbers or formatting alignment), so I can gradually improve my skills.
 - Learning Basic Math:
- 55. As a user, I want a blacksmith NPC to teach me basic math operations (addition, subtraction, multiplication, division) through interactive examples, so I can learn while engaging with the story.
- 56. As a user, I would like to complete an independent quest helping a NPC manage their finances, where I apply math skills to solve real-world-like problems.
- 57. As a user, I want to unlock the in-game shop and earn the achievement "Live for your own" after completing a series of math-based challenges, so I feel a sense of accomplishment.
- 58. As a user, I want the math challenges to scale in difficulty, starting with simple calculations and progressing to multi-step problems, so I can gradually improve my skills.
 - Learning if-else and for/while Loops:
- 59. As a user, I want to learn conditional statements and loops through a maze game where NPCs provide guided examples, so I can practice these concepts in a visual and interactive way.
- 60. As a user, I want to earn rewards for solving maze challenges using if-else statements and loops, so I feel motivated to apply these concepts effectively.
- 61. As a user, I want the looping challenges to scale in difficulty, like with different maze maps, so I can gradually improve my skills.
 - Learning Pointers:
- 62. As a user, I would like a quest involving keys, chests, and addresses to learn the basics of pointers and callback functions, so I can understand how to reference and use memory locations in a practical context.
- 63. As a user, I want an NPC to explain pointers with visual examples (e.g., diagrams showing memory addresses and values), so I can better grasp the concept.
- 64. As a user, I want to unlock pointer-related enchantments (e.g., dynamic item enhancements) after completing the quest, so I feel rewarded for mastering a complex concept.
- 65. As a user, I want to receive a specific achievement or rewards after learning pointers.
- Shop and Blacksmith Features
 - 66. As a user, I want to spend earned money in a shop to purchase blank functional cards, so I can customize my deck and strategy.

67. As a user, I want the blacksmith to enchant functional cards with achieved functions, so my cards can gain new abilities and become more versatile.
68. As a user, I want access to magic cards that provide permanent stamina boosts or extra coin rewards, so I can gain long-term advantages in the game.
69. As a user, I would like to upgrade my functional cards for better performance, so they are more effective during battles.
70. As a user, I want the shop to offer limited-time items (e.g., rare cards, consumables) that rotate daily or weekly, so I feel incentivized to check back regularly.
71. As a user, I would like to trade in unused cards or items for coins or shop credits, so I can make better use of my resources.

Non-functional Requirements

- Cross-Platform Compatibility

Our game will run seamlessly on compatible systems. The Unity frontend will dynamically adjust to various screen sizes and resolutions to ensure a consistent and immersive user experience across all platforms.

- Security

The backend, implemented in **Node.js**, will use **HTTPS** for secure data transmission. User data will be protected with **basic encryption** before storage in **MongoDB**. Only authenticated users can access their own data.

- Performance

The game will ensure the following performance benchmarks:

- The **main menu** will load within **6 seconds** on devices meeting minimum hardware requirements.
 - **Gameplay interactions** (e.g., playing a card, opening a menu, or navigating the map) will respond within **3 seconds** to maintain a smooth user experience.
 - Background tasks, such as saving progress or updating inventory, will execute within **500 milliseconds** under normal load conditions.
- Maintainability

The system will be designed for easy maintenance and future expansion:

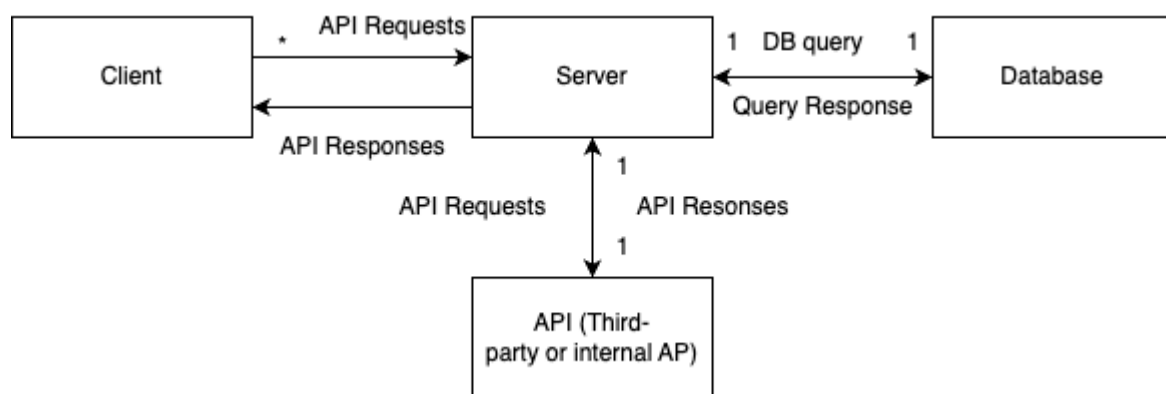
- **Frontend and backend separation** will allow independent updates and debugging, ensuring that new features can be added with minimal disruption to existing functionality.
- The backend will follow a **modular design**, with key features (e.g., authentication, inventory management, shop, and battles) implemented as independent modules or services.
- Usability:

Compared to other existing similar applications, the usability of Bareturn0; is designed to provide an intuitive and seamless gaming experience, focusing on a clean and visually appealing interface that balances functionality with aesthetics. From the moment players launch the game, they are greeted with a polished loading animation that can be skipped for convenience, ensuring a smooth entry into the main menu, where all essential functions such as account management, settings, and progress tracking are easily accessible. The game emphasizes clarity and simplicity, with minimal clutter, intuitive icons, and clear visual cues to guide players through every interaction. Real-time updates, such as health bars, stamina meters, and card pile states, keep players informed without overwhelming them, while features like customizable save slots, deck-building tools, and personalized profiles enhance user engagement and adaptability. The game reinforces motivation through immediate feedback, such as on-screen notifications for achievements and rewards, creating a satisfying sense of progression. By combining intuitive navigation, strategic customization, and visually engaging elements, Bareturn0; ensures an immersive and user-friendly experience that appeals to both new and experienced players.

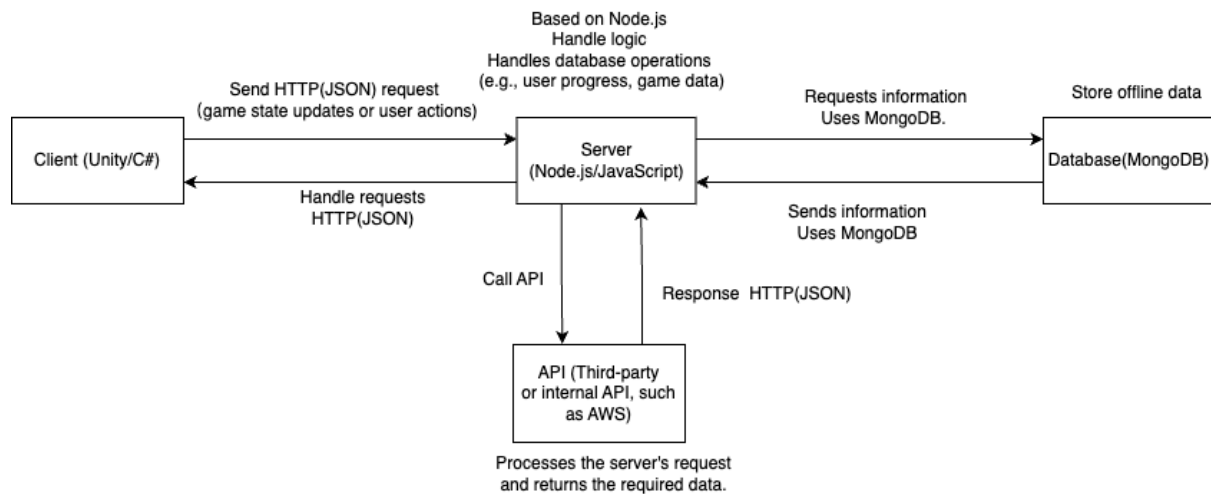
Design Outline

High-Level Overview

This diagram illustrates a typical client-server architecture and the interactions between its components. The Client sends API Requests to the Server, which processes the requests and returns API Responses. The Server also interacts with the Database by sending DB Queries to retrieve or update user data, and the Database responds with Query Responses. Additionally, the Server communicates with a Third-party or Internal API by sending API Requests and receiving API Responses. This architecture ensures efficient communication between the Client, Server, Database, and Third-party APIs while maintaining modularity and scalability.



Detailed Overview



Clients:

1. The client provides users with an interactive game interface developed using Unity and C#. It handles all user interactions, such as progressing through levels, managing character actions, and receiving feedback for achievements or rewards.
2. The client sends HTTP requests (via API calls) to the server to fetch or send data related to player progress, game state, achievements, and card configurations.
3. The client interprets structured responses from the server (in JSON format) and updates the user interface in real time, such as showing updated game progress, displaying rewards, or notifying users about in-game achievements and events.

Server:

1. The server, built with Node.js, acts as the central logic processor for the game. It receives and processes client requests related to game progress, user data, card mechanics, and achievements.
2. The server interacts with the database to retrieve, update, or delete records based on the client's requests, such as player statistics, saved progress, and card deck configurations.
3. The server also integrates with third-party APIs, such as AWS, to handle tasks like cloud storage, asset delivery, and analytics. Processed data is sent back to the client in structured JSON responses for seamless interaction.

Database:

1. The database, implemented using MongoDB, stores all game-related data, including player profiles, game progress, achievements, card configurations, and level unlock statuses.
2. It responds to queries from the server to deliver requested data, such as current player progress, available cards, or unlocked levels. Additionally, it updates its records whenever the user completes a game level, unlocks new achievements, or modifies their game settings.
3. The database ensures consistency and reliability in storing offline data for players, enabling smooth retrieval even during server downtime or disruptions.

API:

1. The API acts as a bridge between the client and server, facilitating the exchange of data and ensuring seamless interaction between different components. The APIs also connect the server with external services like AWS to expand functionality
 - a. **Game Progress API:**

This API allows the client to send and retrieve information about the player's progress, including completed levels, unlocked cards, and achievements. It interacts with the server and database to ensure real-time updates and accurate progress tracking.
 - b. **Card Mechanics API:**

The card mechanics API handles requests related to the player's card deck, such as adding new cards, upgrading existing ones, or shuffling decks during gameplay. It ensures that the server processes the card-related logic and updates the database with changes.
 - c. **Third-party API (AWS):**

The AWS API facilitates cloud-based operations, such as storing user-generated assets, delivering static content, and maintaining backups. It processes server requests and responds with the required data to ensure scalability and reliability

Design Issues

Functional Issues

1. Do users need to log in to use our application?

Option 1: No login required.

Option 2: Allow the creation of login ID using Google, Facebook, etc.

Option 3: Create a unique username and password.

Decision: To ensure personalization and progress tracking, users must create a unique account with a username and password. This allows us to store achievements, game progress, and personalized settings in our database securely.

2. What programming language will be used for teaching?

Option 1: Python

Option 2: C

Option 3: Java

Decision: C is chosen as the teaching language due to its foundational role in programming, widespread usage in system-level programming, and its ability to help learners understand fundamental computer science concepts such as memory management and pointers.

3. How should we integrate C language learning into the game?

Option 1: Provide static tutorials and code examples.

Option 2: Teach C language through interactive challenges with real-time feedback.

Option 3: Integrate C language learning into an RPG-based game background.

Decision: We chose Option 3, embedding C language learning within an RPG framework. Players will complete game tasks that require writing and debugging C code, receiving immediate feedback and rewards. This method enhances

engagement, improves comprehension, and makes learning programming more interactive and enjoyable.

4. What is the game structure?

Option 1: Players progress solely through coding challenges without a card-based system.

Option 2: Players collect and upgrade cards based on their game progress but without learning integration.

Option 3: Players earn stronger cards by completing mini-games that teach C language concepts, using those cards to advance in the main game.

Decision: We chose Option 3, where the game revolves around a card-based system. Players engage in mini-games to learn C programming concepts, and success in these challenges rewards them with more powerful cards. These cards are then used in strategic battles in the main game, ensuring that learning directly contributes to progression. This approach balances education and entertainment, motivating players to master coding concepts while enjoying RPG-style gameplay.

5. How should mini-games be designed to teach different C language concepts?

Option 1: Use a single game format for all C language concepts.

Option 2: Use different types of mini-games tailored to specific C language topics.

Option 3: Allow players to choose between different learning styles (text-based tutorials or mini-games).

Decision: We chose Option 2, where each mini-game is designed to teach a specific C language concept. For example, a maze game could teach loops, a puzzle-solving game could introduce conditionals, and a pointer-based quest could explain memory management. This approach ensures an engaging and structured learning process, helping users grasp different C programming topics in a hands-on, interactive manner.

Non-Functional Issues

1. Which frontend framework will Bareturn0; use?

Option 1: Unity

Option 2: Unreal Engine

Option 3: Custom-built framework

Decision: Unity is the best choice due to its cross-platform capabilities, extensive community support, and flexibility for implementing RPG mechanics and game physics. It allows for easier game development, scalability, and rapid iteration. Moreover, Unity provides robust tools for 2D and 3D rendering, making it suitable for our immersive game environment.

2. Which backend will Baretun0; use?

Option 1: Django (Python)

Option 2: Node.js (JavaScript)

Option 3: Ruby on Rails

Decision: Node.js was selected for its high concurrency, event-driven architecture, and real-time data handling capabilities. It seamlessly integrates with MongoDB and offers a lightweight, efficient solution for managing game state, player interactions, and dynamic content updates.

3. What cloud service will Baretun0; use?

Option 1: AWS

Option 2: Google Cloud

Option 3: Microsoft Azure

Decision: AWS was chosen for its flexibility, reliability, and strong cloud infrastructure. AWS's global network and cost-effective pricing ensure stable performance and smooth player experience, making it a suitable choice for our project.

4. Which kind of database will Baretun0; use?

Option 1: SQL-based database (e.g., MySQL, PostgreSQL).

Option 2: NoSQL-based database (e.g., MongoDB).

Option 3: Hybrid approach combining SQL and NoSQL.

Decision: We chose MongoDB, a NoSQL database, for its flexibility, scalability, and ability to handle unstructured and semi-structured data efficiently. Given that Bareturn0; involves dynamically storing player progress, game states, and educational content, MongoDB's document-based structure provides an optimal solution. Additionally, MongoDB integrates well with Node.js, allowing seamless interaction between the backend and database, improving performance and real-time data management.

5. What platform will Bareturn0; prioritize for development?

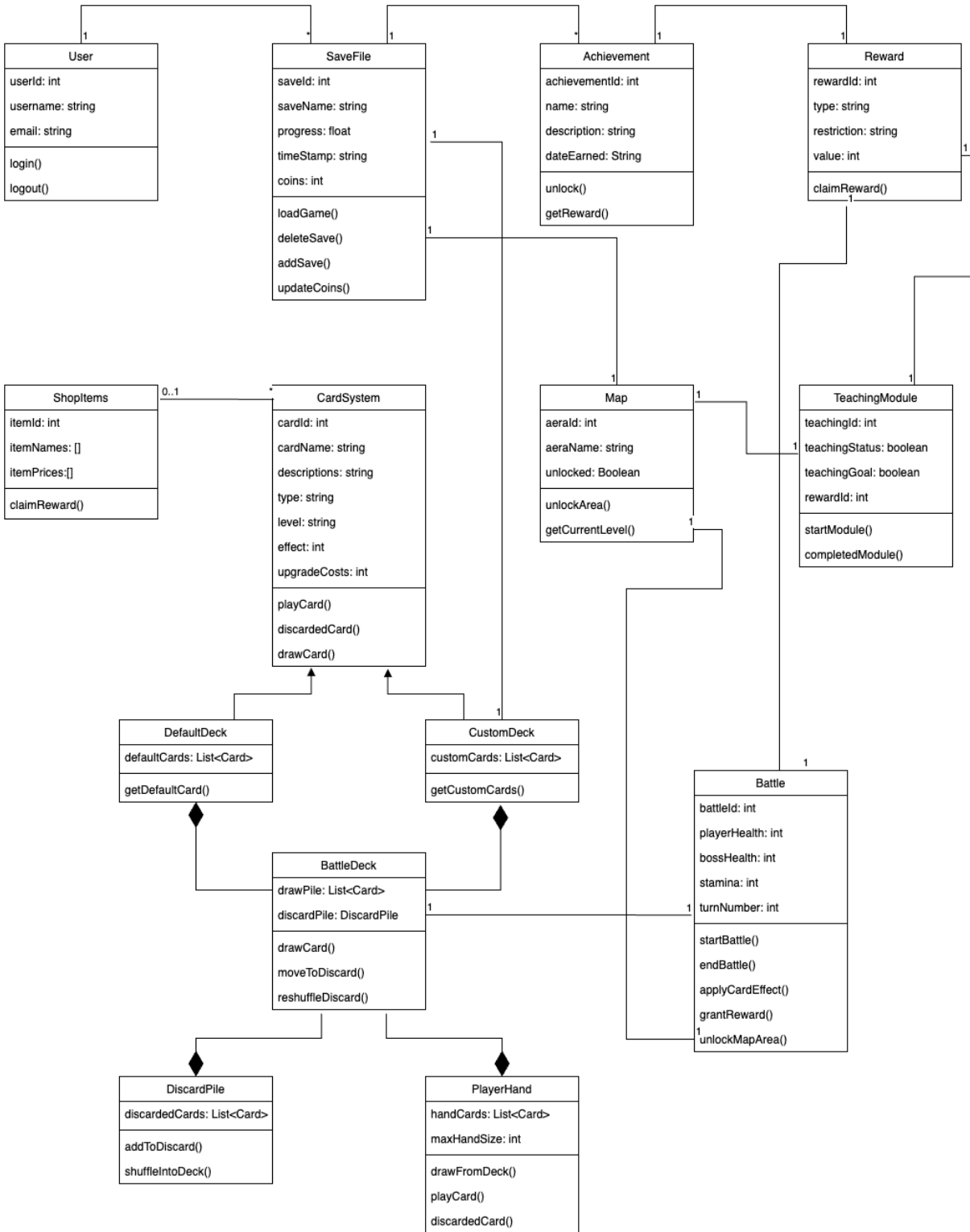
Option 1: Develop for PC first, then expand to other platforms.

Option 2: Develop simultaneously for PC and mobile.

Option 3: Focus on mobile-first development.

Decision: We chose Option 1, prioritizing PC development first to ensure a stable and fully functional experience before expanding to other platforms. Developing on PC first allows for easier debugging, better performance optimization, and a more comprehensive UI/UX design. Once the core game mechanics and educational features are well-implemented, we will assess feasibility for mobile or console expansion based on user demand.

Class Design



Class Description

User

- **Attributes:**
 - `userId`: Unique identifier for each user.
 - `username`: Name of the user.
 - `email`: Email associated with the user.
- **Methods:**
 - `login()`: Handles user authentication.
 - `logout()`: Logs the user out of the system.

SaveFile

- **Attributes:**
 - `saveId`: Unique ID for each save file.
 - `saveName`: User-defined name for the save file.
 - `progress`: Represents game progress percentage.
 - `timeStamp`: Records the last save time.
 - `coins`: In-game currency stored for the user.
- **Methods:**
 - `loadGame()`: Loads the selected save file.
 - `deleteSave()`: Deletes a specific save file.
 - `addSave()`: Creates a new save file.
 - `updateCoins()`: Updates the amount of in-game currency.

Achievement

- **Attributes:**
 - `achievementId`: Unique ID for the achievement.
 - `name`: Name of the achievement.
 - `description`: Details about the achievement.
 - `dateEarned`: The date when the achievement was unlocked.
- **Methods:**
 - `unlock()`: Unlocks an achievement upon meeting conditions.
 - `getReward()`: Retrieves the reward associated with the achievement.

Reward

- **Attributes:**
 - `rewardId`: Unique ID for each reward.
 - `type`: The type of reward (coins, items, etc.).
 - `restriction`: Restrictions or conditions for claiming the reward.
 - `value`: The numerical value of the reward.
- **Methods:**
 - `claimReward()`: Allows the user to claim the reward.

ShopItems

- Attributes:
 - itemId: Unique ID for the shop item.
 - itemNames: List of available items in the shop.
 - itemPrices: Prices corresponding to each item.
- Methods:
 - claimReward(): Allows users to purchase an item with in-game currency.

CardSystem

- Attributes:
 - cardId: Unique identifier for each card.
 - cardName: Name of the card.
 - description: Description of the card's effect.
 - type: Type of the card (attack, defense, utility, etc.).
 - level: The level of the card, indicating power.
 - effect: The effect value of the card.
- upgradeCosts: The cost to upgrade the card.
- Methods:
 - playCard(): Executes the card's effect in battle.
 - discardCard(): Moves the card to the discard pile.
 - drawCard(): Draws a new card from the deck.

DefaultDeck

- Attribute:
 - defaultCards: The set of predefined cards available.
- Method:
 - getDefaultCard(): Returns a card from the default deck.

CustomDeck

- Attribute:
 - customCards: A set of cards customized by the player.
- Method:
 - getCustomCards(): Retrieves a card from the custom deck.

BattleDeck

- Attributes:
 - drawPile: Cards available for drawing.
 - discardPile: Reference to the discard pile.

- **Methods:**
 - `drawCard()`: Draws a card from the draw pile.
 - `moveToDiscard()`: Moves a played card to the discard pile.
 - `reshuffleDiscard()`: Shuffles the discard pile back into the draw pile.

DiscardPile

- **Attribute:**
 - `discardedCards`: Cards that have been used and discarded.
- **Methods:**
 - `addToDiscard()`: Adds a card to the discard pile.
 - `shuffleIntoDeck()`: Moves all discarded cards back into the battle deck.

PlayerHand

- **Attributes:**
 - `handCards`: Cards currently in the player's hand.
 - `maxHandSize`: The maximum number of cards a player can hold.
- **Methods:**
 - `drawFromDeck()`: Draws a card from the battle deck.
 - `playCard()`: Plays a card from the player's hand.
 - `discardCard()`: Moves a card from the player's hand to the discard pile.

Map

- **Attributes:**
 - `areaid`: Unique ID for each map area.
 - `areaName`: string: Name of the map area.
 - `unlocked`: Boolean: Whether the area is accessible.
- **Methods:**
 - `unlockArea()`: Unlocks a new area.
 - `getCurrentLevel()`: Retrieves the current level within the area.

Battle

- **Attributes:**
 - `battleId`: Unique ID for the battle instance.
 - `playerHealth`: Player's health points.
 - `bossHealth`: Boss's health points.
 - `stamina`: The stamina available for playing cards.
 - `turnNumber`: The number of turns taken in battle.
- **Methods:**
 - `startBattle()`: Initiates a new battle.
 - `endBattle()`: Ends the battle and determines victory/loss.

- `applyCardEffect()`: Applies the effect of a played card.
- `grantReward()`: Grants rewards upon battle completion.
- `unlockMapArea()`: Unlocks a new map area upon victory.

Teaching Module:

- **Attributes:**
 - `teachingId`: Unique ID for the teaching instance(related to topic and further scripts).
 - `teachingStatus`: Status of whether the player has started the learning module or not.
 - `teachingGoal`: Whether the player has finished all tasks or not.
 - `rewardId`: The reward of completing this module.
- **Methods:**
 - `startModule()`: Initiates a learning module.
 - `completeModule()`: Mark the module has been completed.

Interaction Between Classes:

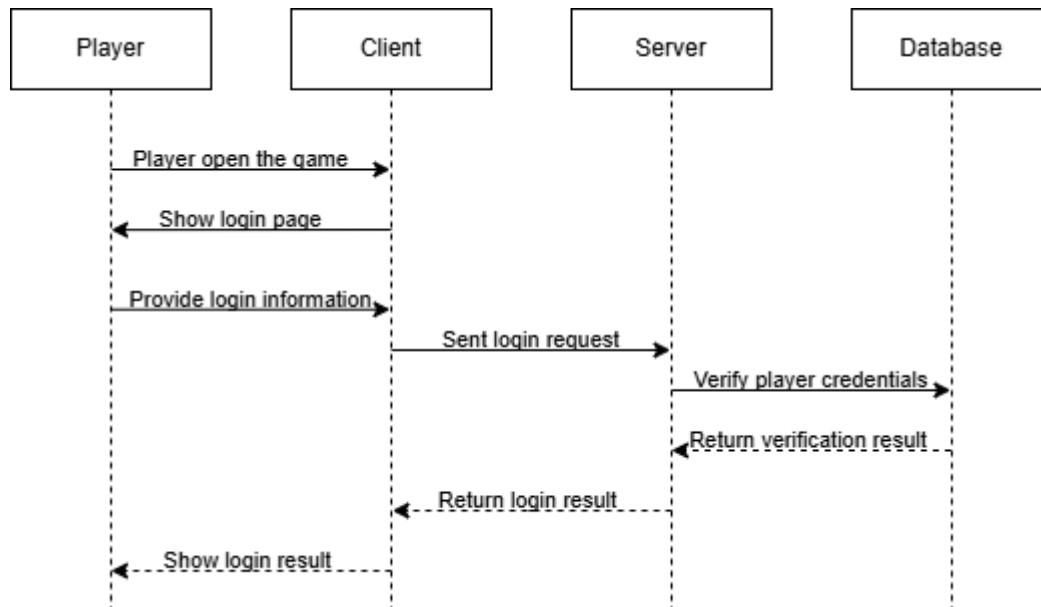
- **User Saves and Loads Game Progress :**
 - **Classes Involved:** User, SaveFile, Achievement, Reward
 - User selects a SaveFile to load game progress. (`loadGame()`)
 - SaveFile retrieves:
 - `progress`: Current game progress percentage.
 - `coins`: The player's in-game currency.
 - `achievements`: Unlocked achievements.
 - If achievements are unlocked, `Achievement.getReward()` is called.
 - `Reward.claimReward()` is executed, granting items, coins, or other benefits to the user.
 - User can update the save state at any point(except during battles) using `SaveFile.updateCoins()`, `addSave()`, or `deleteSave()`.
- **Managing Card Decks**
 - **Classes Involved:** CardSystem, DefaultDeck, CustomDeck, BattleDeck, DiscardPile, PlayerHand
 - **Game Setup:**
 - `DefaultDeck.getDefaultCard()` and `CustomDeck.getCustomCards()` provide cards to form `BattleDeck.drawPile`.
 - BattleDeck is created by combining DefaultDeck and CustomDeck cards, chosen by the player.
 - The player starts with a PlayerHand drawn from BattleDeck.
 - **During Battle:**

- PlayerHand.drawFromDeck() retrieves cards from BattleDeck.drawPile.
 - When a player plays a card, PlayerHand.playCard() sends the card to Battle.applyCardEffect().
 - Used cards are sent to DiscardPile.addToDiscard().
 - When BattleDeck.drawPile is Empty:
 - BattleDeck.resuffleDiscard() calls DiscardPile.shuffleIntoDeck().
 - discardedCards from DiscardPile are moved back into BattleDeck.drawPile.
 - The cycle continues until the battle ends.
- Battle Execution
 - Classes Involved: Battle, PlayerHand, BattleDeck, DiscardPile, Map, Reward
 - Starting a Battle:
 - Battle.startBattle() initializes player health, boss health, stamina, and turn count.
 - PlayerHand.drawFromDeck() fills the player's hand for the turn.
 - Player Turn:
 - The player selects a card(or multiple cards) from PlayerHand and calls playCard().
 - Battle.applyCardEffect() processes the action:
 - If attack, the enemy loses health.
 - If buff or healing, the player gains benefits.
 - The used card is moved to DiscardPile.
 - Player draws new cards from BattleDeck.drawPile.
 - Enemy Turn:
 - Enemy action occurs.
 - The battle cycle continues.
 - Ending the Battle:
 - If the player wins, Battle.grantReward() is executed:
 - Reward.claimReward() gives coins or items.
 - Battle.unlockMapArea() unlocks new areas in Map.
- Buying Cards and Items in Shop
 - Classes Involved: ShopItems, SaveFile, CustomDeck, CardSystem
 - The player selects an item in ShopItems.
 - The system checks if SaveFile.coins is enough for the purchase.
 - If successful:
 - SaveFile.updateCoins() deducts the price.
 - If the item is a card:
 - CardSystem creates a new card.
 - CustomDeck.getCustomCards() adds it to the player's collection.
- Unlocking New Areas

- Classes Involved: Battle, Map, Achievement
- The player wins a battle (Battle.endBattle()).
- Battle.unlockMapArea() is called:
- Map.unlockArea() marks the new area as accessible.
- If unlocking an area is a milestone, Achievement.unlock() is triggered.
- Achievement.getReward() provides an achievement-based reward.
- Learning System and the Game
 - Classes Involved: TeachingModule, Map, Reward, SaveFile, Achievement
 - Starting a Learning Module:
 - The player selects a TeachingModule and calls startModule().
 - The game marks teachingStatus = true, indicating the module has started.
 - Completing the Learning Module:
 - Once the player finishes all required tasks, completeModule() is called.
 - teachingGoal = true, marking the module as completed.
 - Granting Rewards and Updating Progress:
 - TeachingModule.completeModule() triggers Reward.claimReward().
 - If applicable, Achievement.unlock() is called.
 - SaveFile.updateProgress() updates the save file, reflecting learning progress.
 - The game may unlock new areas (Map.unlockArea()).
- Unlocking New Areas After Learning
 - Classes Involved: TeachingModule, Map, Battle
 - The player completes TeachingModule.completeModule(), marking it as finished.
 - If the module is tied to a new area, Map.unlockArea() is called.
 - The newly unlocked area may contain:
 - New TeachingModule lessons.
 - New battles (Battle.startBattle()).
 - Additional rewards (Reward.claimReward()).

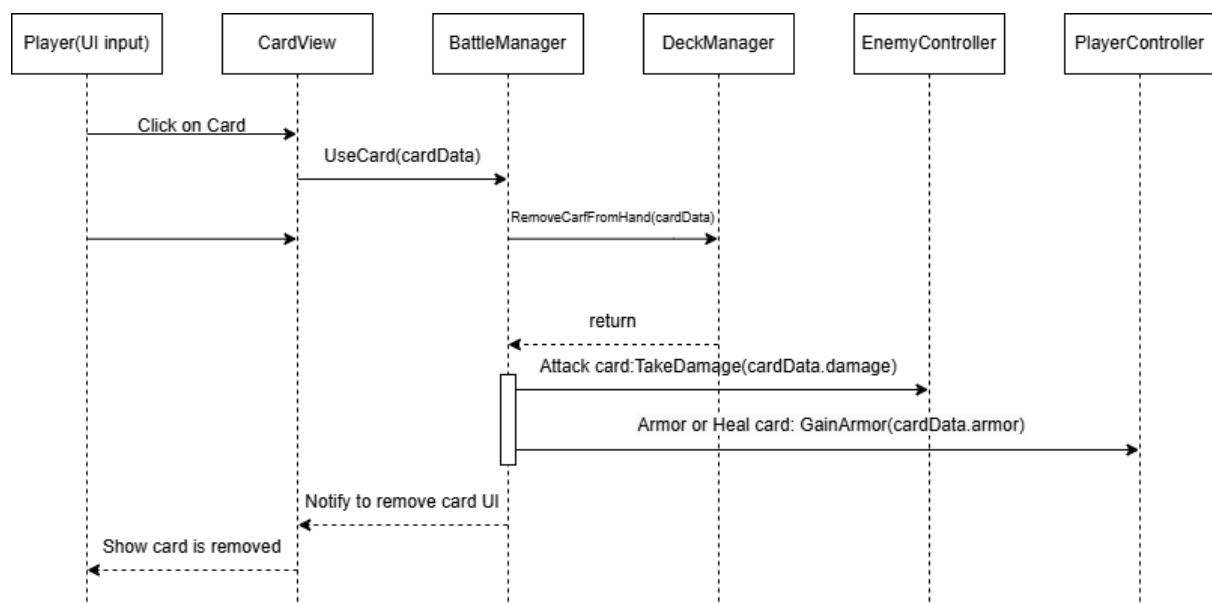
Sequence of Events Overview

- User login:



When the user opens the game, the client displays the login page, prompting the user to provide their credentials. These credentials are then sent to the server, which validates them by querying the database. The database returns the verification result, allowing the server to determine whether access is granted or denied. Finally, the server sends the outcome back to the client, which updates the user interface to show the login result.

- Playing Card:



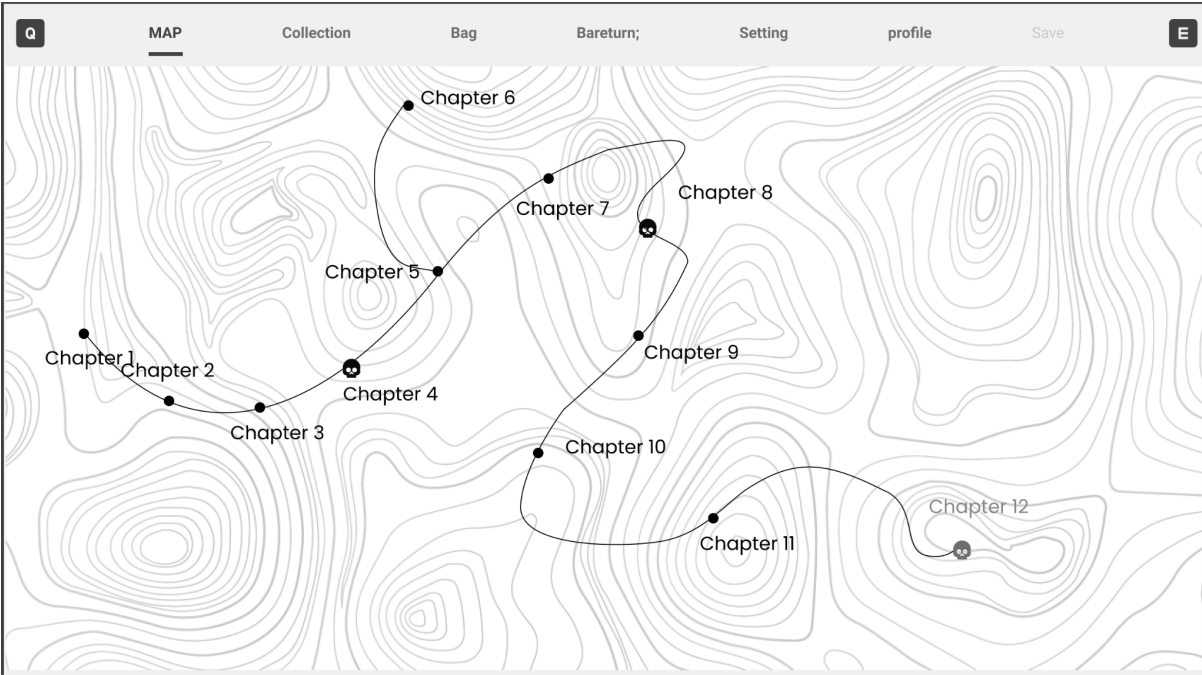
This diagram depicts the sequence of event that will happen when the player is trying to use a card in the card game. When the player clicks on a card, the CardView sends the card's data to the BattleManager to execute its effect. The BattleManager requests the DeckManager to remove that card from the player's hand and then processes the card's action—whether dealing damage to an enemy or granting armor to the player. Once the effect is resolved, the CardView is notified to update the UI and remove the card visually, reflecting that the card has been played and is no longer available in the player's hand.

UI Mockups



Load Game

Character name	Game Progress	Save Time
Charlie Fang	printf	01/30/2025
Xinyi Guan	calculate	01/26/2025
Xinyi Wu	pointer	01/25/2025
Yuchuan Huang	loop	08/09/2024
barrrrrrrrr	99%	04/22/2024
return;	100%	10/10/1010



Q

MAPCollectionBagBareturn;SettingprofileSave

E

5 / 20

Person You Know Who

01/30/2024

XXXXX

XX/XX/XXXX

XXXXX

XX/XX/XXXX

XXXXX

XX/XX/XXXX

XXXXX

XX/XX/XXXX

?

?

?

?

?



Q

MAP

Collection

Bag

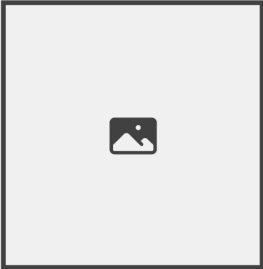
Bareturn;

Setting

profile

Save

E





49 / 50

NAME

barrrrrrrrr

USER NAME

Bareturn;

BIRTHDAY

05/02/2025

PASSWORD

Change password

Logout

